МИКРОКОНТРОЛЛЕРЫ СЕМЕЙСТВА ATMEGA

Arduino — торговая марка аппаратно - программных средств, для построения простых систем автоматики и робототехники, ориентированная на непрофессиональных пользователей. Программная часть состоит из бесплатной программной оболочки (IDE) для написания программ, их компиляции и программирования аппаратуры. Аппаратная часть представляет собой набор смонтированных печатных плат, продающихся как официальным производителем, так и сторонними производителями. Полностью открытая архитектура системы позволяет свободно копировать или дополнять линейку продукции Arduino.

Под торговой маркой Arduino выпускается несколько плат с микроконтроллером и платы расширения (так называемые «шилды»). Большинство плат с микроконтроллером снабжены минимально необходимым набором обвязки для нормальной работы микроконтроллера (стабилизатор питания, кварцевый резонатор, цепочки сброса и т. п.).

Конструктив

Arduino и Arduino - совместимые платы спроектированы таким образом, чтобы их можно было при необходимости расширять, добавляя в устройство новые компоненты. Эти платы расширений подключаются к Arduino посредством установленных на них штыревых разъёмов. Существует ряд плат с унифицированным конструктивом, допускающим конструктивно жесткое соединение процессорной платы и плат расширения в стопку через штыревые линейки. Кроме того, выпускаются платы уменьшенных габаритов (например, Nano) и специальных конструктивов для задач робототехники. Независимыми производителями также выпускается большая гамма всевозможных датчиков и исполнительных устройств, в той или иной степени совместимых с базовым конструктивом Arduino.

В концепцию Arduino не входит корпусной или монтажный конструктив. Разработчик выбирает метод установки и механической защиты плат самостоятельно. Сторонними производителями выпускаются наборы робототехнической электромеханики, ориентированной на работу совместно с платами Arduino.

Микроконтроллер

Микроконтроллеры для Arduino отличаются наличием предварительно прошитого в них загрузчика (bootloader). С помощью этого загрузчика пользователь загружает свою программу в микроконтроллер без использования традиционных отдельных аппаратных программаторов.

Загрузчик соединяется с компьютером через интерфейс USB (если он есть на плате) или с помощью отдельного переходника UART-USB. Поддержка загрузчика встроена в Arduino IDE и выполняется в один щелчок мыши.

На случай затирания загрузчика или покупки микроконтроллера без загрузчика разработчики предоставляют возможность прошить загрузчик в микроконтроллер самостоятельно. Для этого в Arduino IDE встроена поддержка нескольких популярных дешевых программаторов, а большинство плат Arduino имеет штыревой разъем для внутрисхемного программирования. Используемый микроконтроллер ATmega2560 изображен на плате Arduino (рисунок 1).

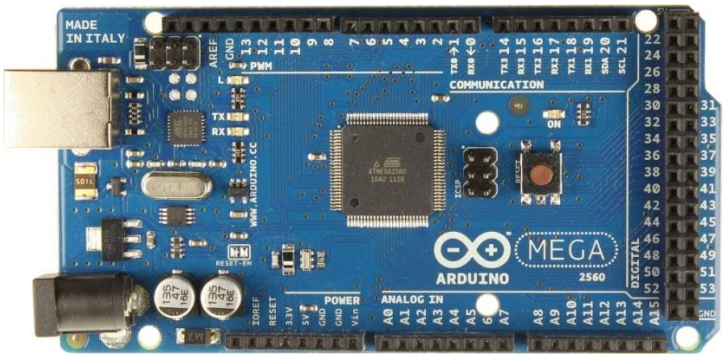

Рисунок 1 - Изображение платы Arduino ATmega2560

Распиновка платы

Порты ввода-вывода микроконтроллеров оформлены в виде штыревых линеек. Никакого буферизирования, защиты, конвертации уровней или подтяжек, как правило, нет. Микроконтроллеры питаются от 5В или 3,3В, в зависимости от модели платы. Соответственно порты имеют такой же размах допустимых входных и выходных напряжений. Программисту доступны некоторые специальные возможности портов ввода-вывода микроконтроллеров, например, широтно-импульсная модуляция (ШИМ), аналогово-цифровой преобразователь (АЦП), интерфейсы UART, SPI, I2C. Количество и возможности портов ввода-вывода определяются конкретным вариантом микропроцессорной платы. Изображение распиновки используемой платы представлено на рисунке 2.

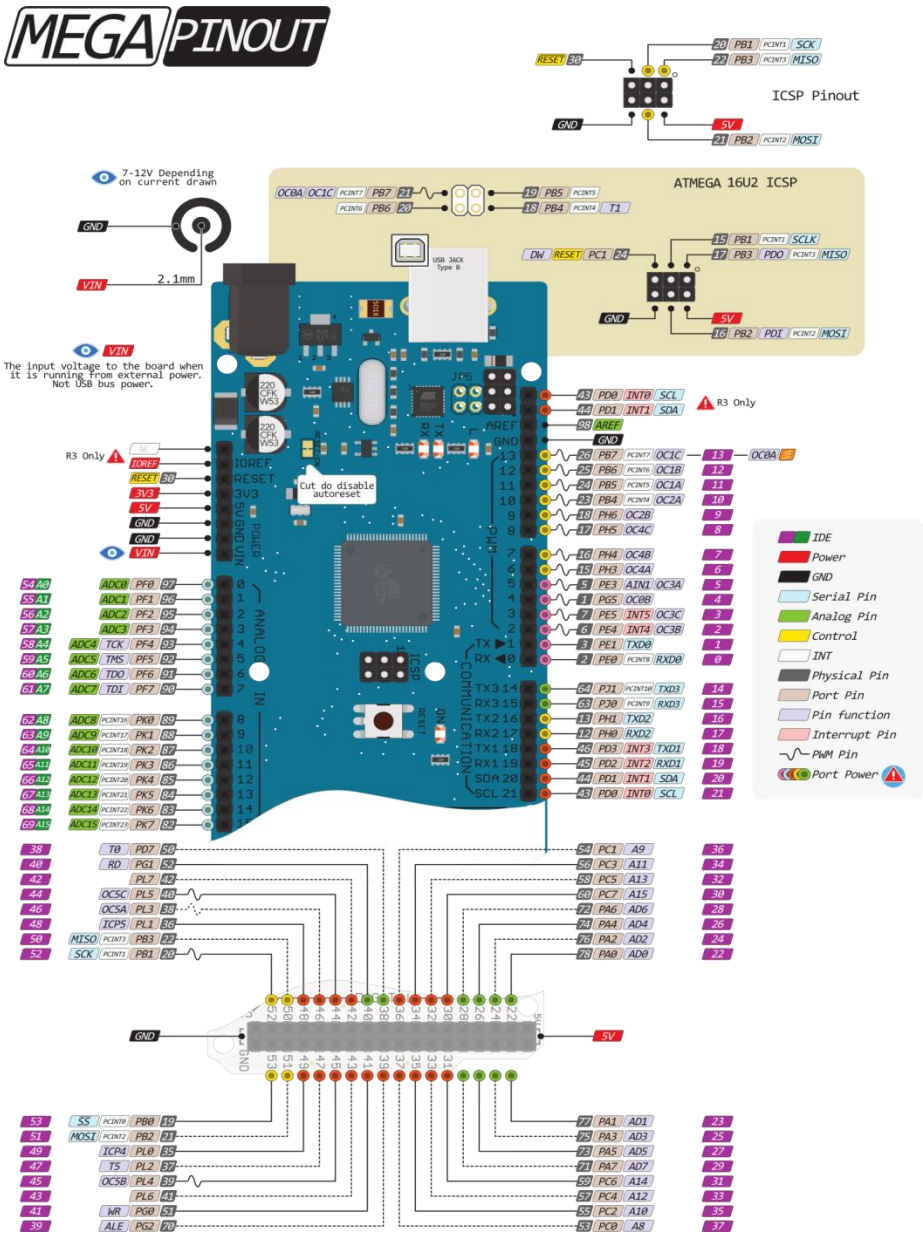

Рисунок 2 – Изображение распиновки платы Arduino ATmega2560

Концепция программирования

Программирование ведется целиком через собственную программную оболочку (IDE), бесплатно доступную на сайте Arduino. В этой оболочке имеется текстовый редактор, менеджер проектов, препроцессор, компилятор и инструменты для загрузки программы в микроконтроллер. Оболочка написана на Java на основе проекта Processing, разработана под такие операционный системы, как Windows, Mac OS X и Linux. Оболочка программы изображена на рисунке 3.

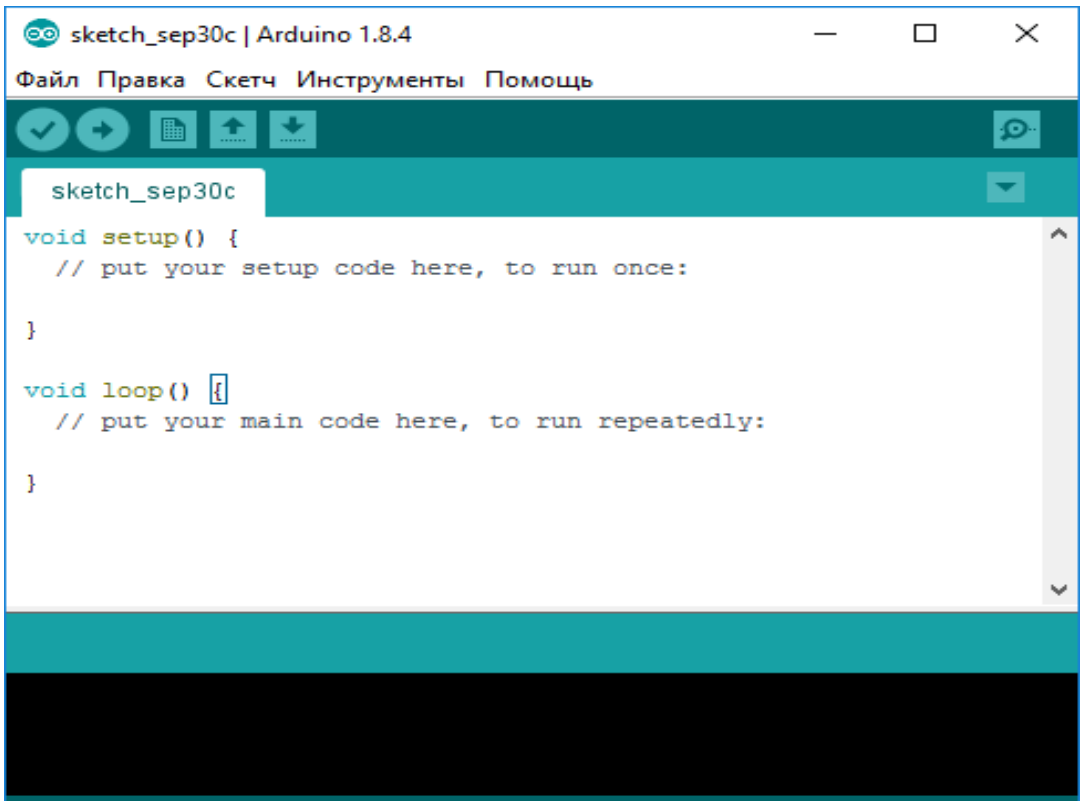

Рисунок 3 - Скриншот среды разработки Arduino IDE

Датчик шума на базе усилителя LM393

Аналоговый датчик звука RKP-SS-LM393-4-Pin (Рисунок 4) предназначен для совместного использования с устройствами, использующими платформу Arduino.

Модуль звуковых колебаний или попросту датчик звука представляет собой небольшую печатную плату, на которой установлены высокочувствительный электретный микрофон, микросхема усилителя сигнала LM393, переменный резистор для регулировки чувствительности, а также элементы для согласования входа/выхода и питания.

Датчик звука с цифровым выходом реагирует на звук определённой громкости. Звуковой сенсор снабжен специальным штырьковым 4-х пиновый разъемом (типа «папа») для подключения к плате микроконтроллера Arduino.

Наилучшая дистанция для определения звука изменяется подбором чувствительности сенсора. Для этой цели на плате звукового модуля RKP-SS-LM393-4-Pin имеется специальный регулятор подстройки чувствительности.

Датчик срабатывает при звуках громкостью выше установленного порога. Модуль датчика звука применяется в приборах содержащих аналоговую или цифровую электронику, в том числе и электронику на базе микроконтроллеров.

Он может служить датчиком присутствия в охранных системах. Также модуль RKP-SS-LM393-4-Pin применяется в автоматике управления освещением. Реагирует на звук шагов, щелчки замка, шум двигателя автомобиля, может включать и выключать свет, включать телевизор, переключать телевизионные каналы, открывать/закрывать жалюзи и многое другое.

Технические характеристики датчика звука RKP-SS-LM393-4-Pin (табл. 1)

Таблица 1 - Характеристика аналогового датчика звука RKP-SS-LM393-4-Pin

Характеристика	Значение
Ток потребления	+3.3 В ~ +5 В
Входное напряжение сигнала	1.4 мА
Тип выходного сигнала	цифровой TTL, аналоговый
Выходное напряжение логической единицы	5 В
Рабочая температура	от 0 ° С ~ + 70 ° С
Размеры	44 х 15 х 14 мм
Вес	3.3 г
Диаметр отверстия для монтажа	3 мм

Определение превышения порога громкости звука происходит с помощью микросхемы компаратора LM393. Настройка порога срабатывания выполняется потенциометром находящимся на плате модуля. Если порог громкости звука превышен, то на выходе D0 появляется высокий уровень напряжения. Уровень напряжения на аналоговом выходе A0 соответствует уровню громкости окружающих шумов. Если применяется микроконтроллер, то возможна программная обработка колебаний уровня напряжения на выходе A0 и сравнение с имеющейся базой в памяти мк колебаний аналогового сигнала, что позволяет распознавать некоторые события происходящие рядом с датчиком.

На плате датчика есть два светодиода обозначенные L1 и L2. Светодиод L1 горит, когда на датчик подается напряжение питания. Светодиод L2 горит, когда датчик улавливает звуковые колебания.

На плате модуля установлена вилка разъема из четырех контактов и нанесены подписи для каждого контакта. Так же датчик имеет 2 выхода: цифровой и аналоговый:

- Вывод с меткой «AO» – аналоговый выходной сигнал (напряжение соответствует уровню громкости окружающих шумов);

- Вывод с меткой «G» (GND) - минус питания;

- Вывод с меткой «+» (VCC) - плюс питания плюс (+3.3V ~ +5V);

- Вывод с меткой «DO» – цифровой выходной сигнал (выход логического сигнала сообщающего о превышении установленного порога).

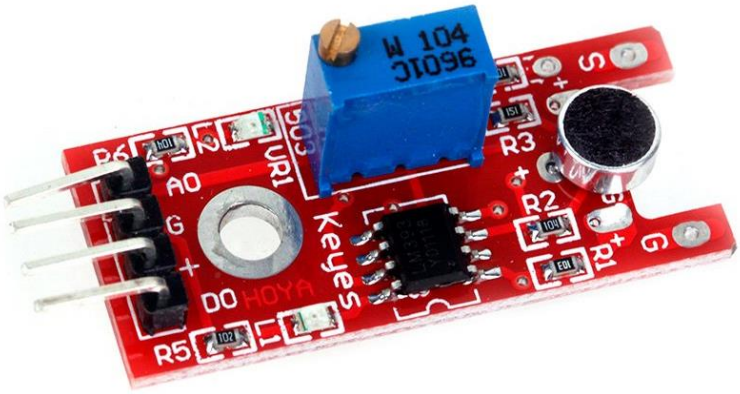

Рисунок 4 – Изображение датчика шума на базе усилителя LM393

Мощная нагрузка (коллекторный электродвигатель)

Коллекторный электродвигатель постоянного тока - вращающаяся электрическая машина постоянного тока, преобразующая электрическую энергию постоянного тока в механическую энергию, у которой хотя бы одна из обмоток, участвующих в основном процессе преобразования энергии, соединена с коллектором. Используемая мощная нагрузка представлена на рисунке 5.

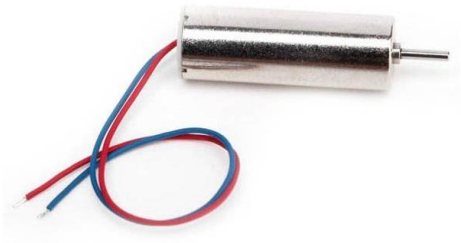

Рисунок 5 – Изображение коллекторного электродвигателя

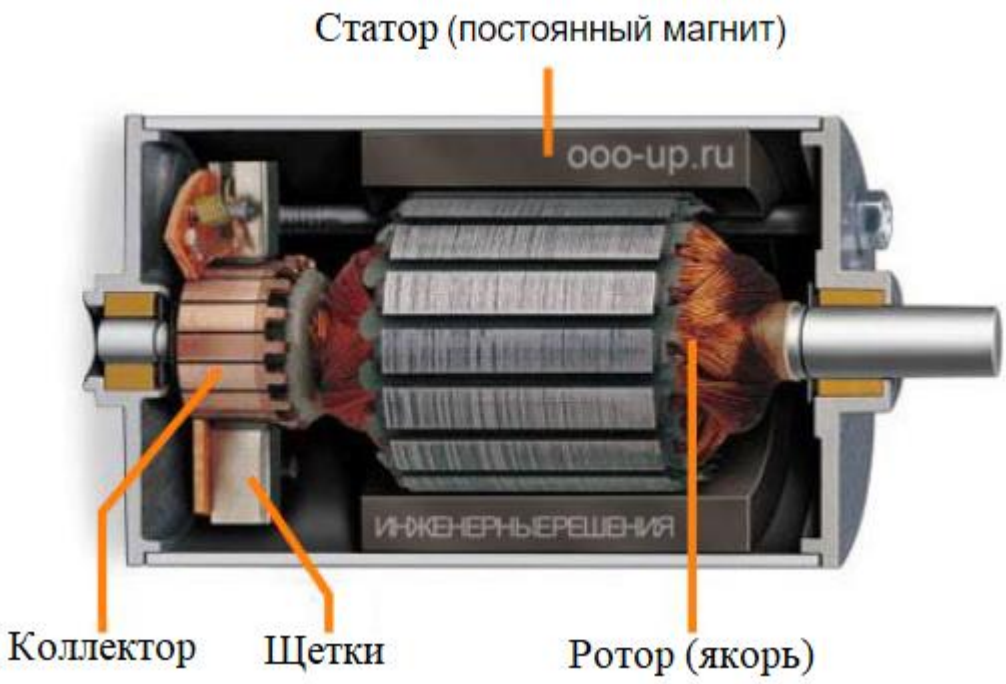

Рисунок 6 – Изображение электродвигателя постоянного тока с постоянными магнитами в разрезе

где:

- ротор — вращающаяся часть электрической машины;

- статор — неподвижная часть двигателя;

- индуктор (система возбуждения) — часть коллекторной машины постоянного тока или синхронной машины, создающая магнитный поток для образования момента. Индуктор обязательно включает либо постоянные магниты либо обмотку возбуждения. Индуктор может быть частью как ротора так и статора. В двигателе, изображенном на рисунке 6, система возбуждения состоит из двух постоянных магнитов и входит в состав статора;

- якорь — часть коллекторной машины постоянного тока или синхронной машины, в которой индуктируется электродвижущая сила и протекает ток нагрузки. В качестве якоря может выступать как ротор так и статор. В двигателе, показанном на рисунке 6, ротор является якорем;

- щетки — часть электрической цепи, по которой от источника питания электрический ток передается к якорю. Щетки изготавливаются из графита или других материалов. Двигатель постоянного тока содержит одну пару щеток или более. Одна из двух щеток соединяется с положительным, а другая — с отрицательным выводом источника питания;

- коллектор — часть двигателя, контактирующая со щетками. С помощью щеток и коллектора электрический ток распределяется по катушкам обмотки якоря.

Принцип работы электродвигателя представлене на рисунке 7.

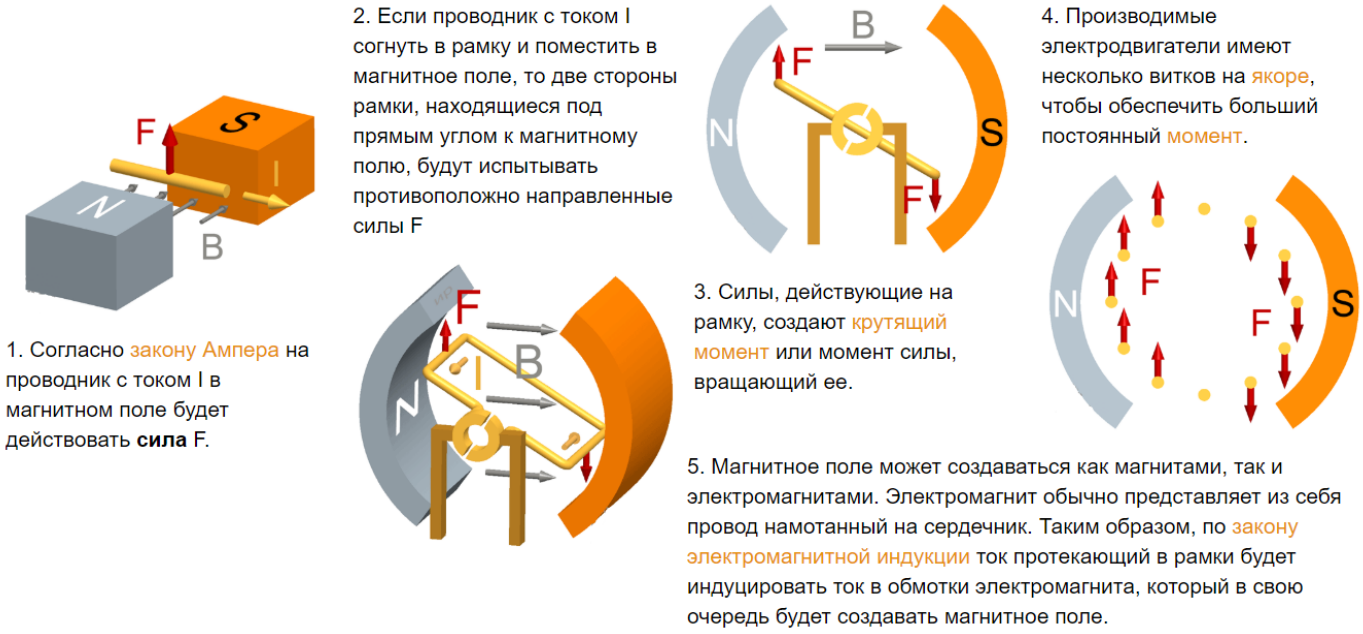

Рисунок 7 – Изображение принципа работы электродвигателя

Модуль управления мотором (Motor driver MX 1616)

Для управления двигателями необходимо устройство, которое бы преобразовывало управляющие сигналы малой мощности в токи, достаточные для управления моторами. Такое устройство называют драйвером двигателей. Используемый драйвер двигателя изображен на рисунке 8.

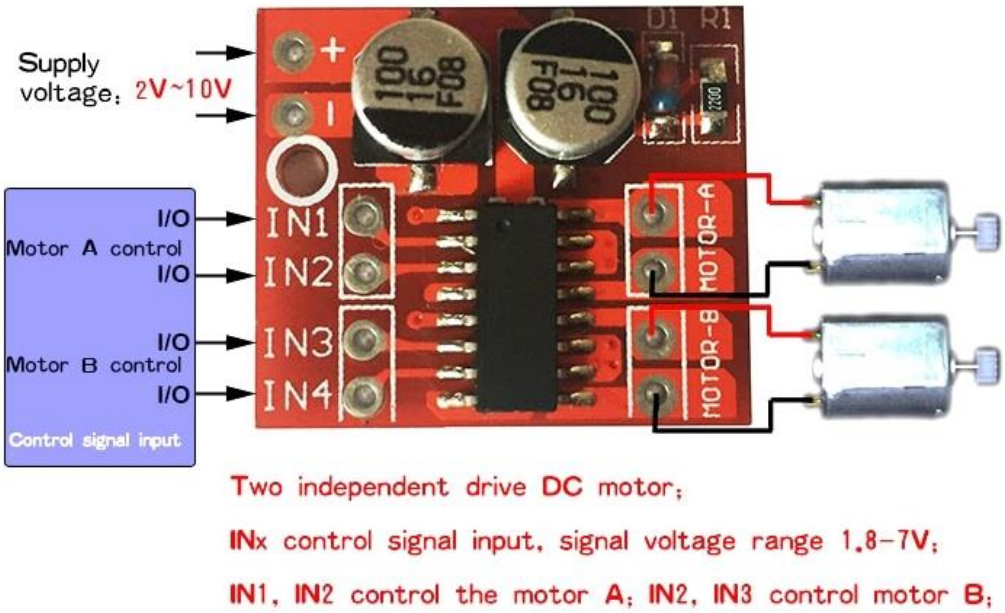

Рисунок 8 – Изображение модуля управления мотором

Технические характеристики драйвера представлены в таблице 2

Таблица 2 - Характеристика Motor driver MX 1616

Характеристика	Значение
Напряжение питания модуля	DC 2V-10 V
Входное напряжение сигнала	DC 1.8-7 V
Рабочий ток	1.5 A
Пиковый ток	до 2,5A
Габариты	24,7 x 21 x 7 мм
Диаметр монтажного отверстия	2 мм

Практически у всех разработчиков устройств на микроконтроллерах, будь то любители или профессионалы, рано или поздно возникает необходимость подключить микроконтроллерный девайс к его «старшему брату», а именно к PC. Вот тогда и встает вопрос, а какой софт использовать для обмена с микроконтроллером, анализировать и обрабатывать полученные от него данные?

На стороне компьютера применяют различные терминальные программы, коих сотни. Но эти программы обеспечивают лишь прием и передачу информации. Как то обрабатывать и визуализировать ее в наглядной форме затруднительно.

Некоторые пишут подобное ПО самостоятельно на каком либо языке программирования (Delphi, C++), наделяя необходимым функционалом. Но эта задача не из легких, нужно знать, помимо самого языка, устройство операционной

системы, способы работы с коммуникационными портами, множество других технических тонкостей, которые отвлекают от главного — реализации алгоритма программы. В общем, быть попутно еще Windows/Unix программистом.

На фоне этих подходов резко отличается концепция виртуальных приборов (vi). LabVIEW является одой из основных продуктов компании National Instruments. Прежде всего надо отметить, что LabVIEW — это аббревиатура, которая расшифровывается как Laboratory Virtual Instrumentation Engineering Workbench. Уже в названии прослеживается ориентация на лабораторные исследования, измерения и сбор данных. Действительно, построить SCADA — систему в LabVIEW несколько проще чем при использовании «традиционных» средств разработки.

LabVIEW — язык графического программирования, в котором для создания приложений используются графические образы (иконки) вместо традиционного текстового кода. От пользователя пакета не требуется знаний языков программирования, но понятие об алгоритме, цикле, выходе по условию и т.п. конечно иметь нужно. Все действия сводятся к простому построению струк - турной схемы приложения в интерактивной графической системе с набором всех необходимых библиотечных образов, из которых собираются объекты, называемые Виртуальными Инструментами (VI). Рабочие окна программы представлены на рисунках 9 и 10.

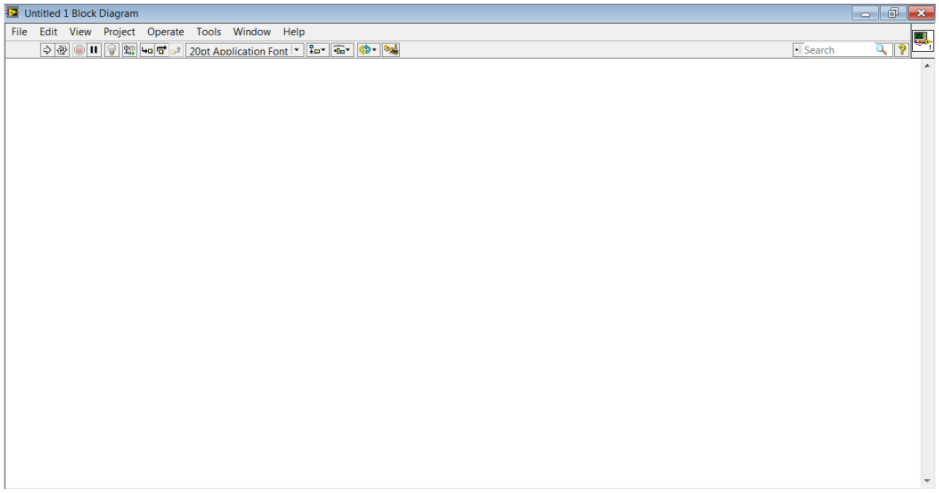

Рисунок 9 – Скриншот среды разработки Labview (Block Diagram)

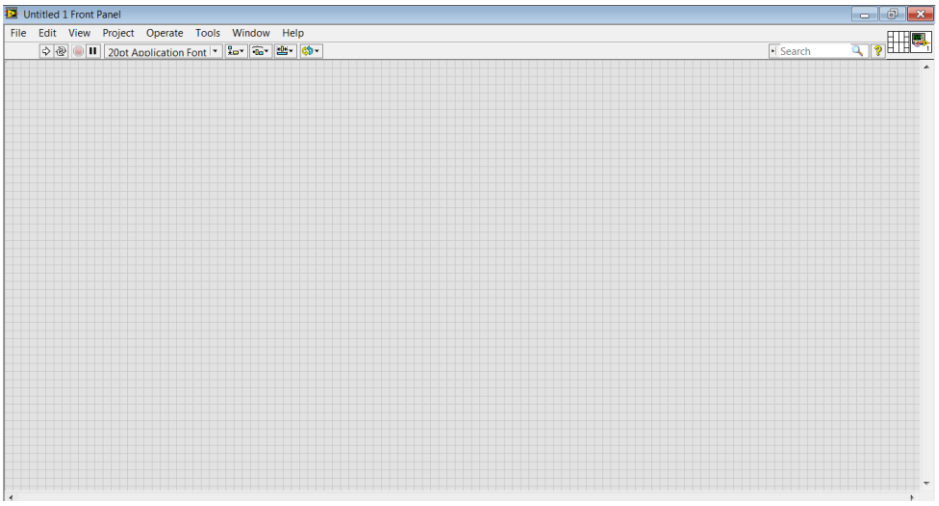

Рисунок 10 – Скриншот среды разработки Labview (Front Panel)

Структурная схема подключения

Составим структурную схему подключения мощной нагрузки, в роли которой выступает коллекторный электродвигатель (Рисунок 11) для определения связей между используемыми устройствами.

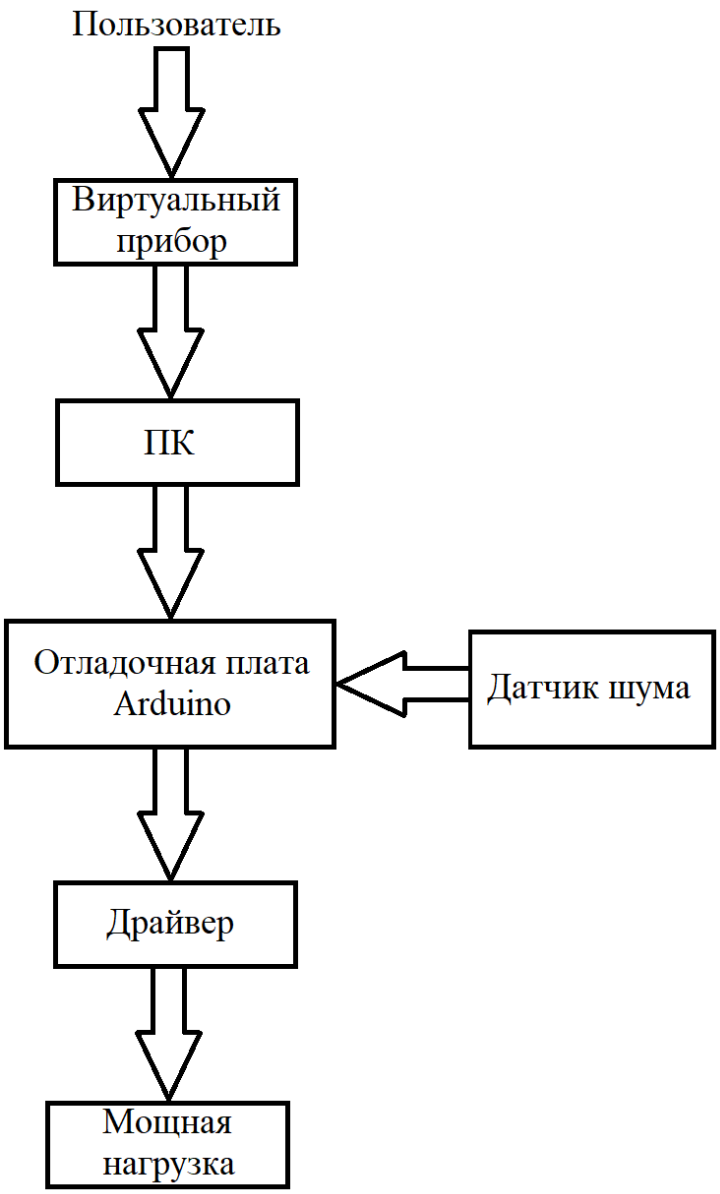

Рисунок 11 – Изображение структурной схемы подключения коллекторного электродвигателя

Схема подключения эллектродвигателя

Основываясь на составленой структурной схеме (рисунок 11), составим схему подключения используемых устройств и датчика (рис. 12).

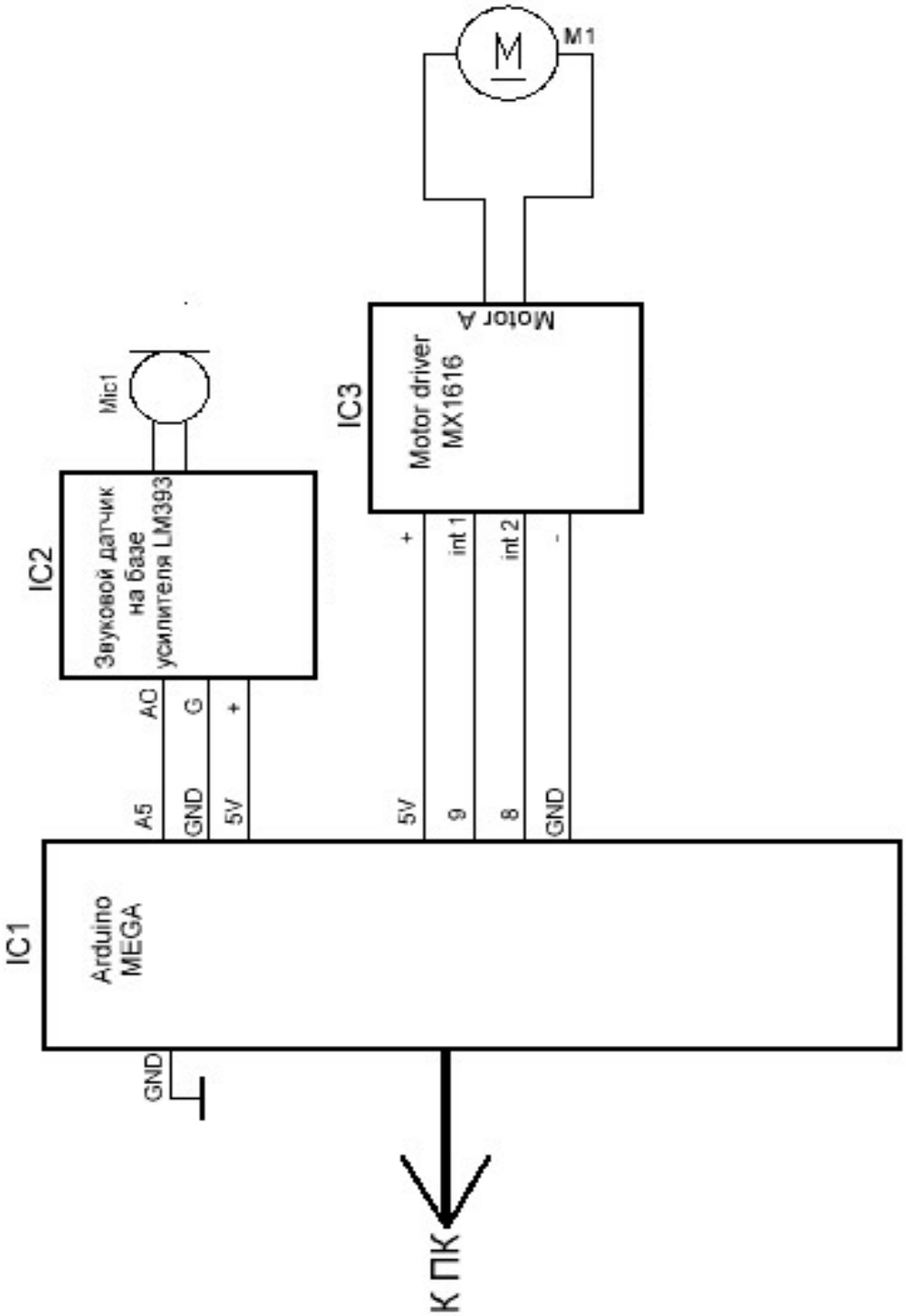

Рисунок 12 – Схема подключения коллекторного электродвигателя

Алгоритм управления

Перед написанием программы реализующей управление коллекторным электродвигателем составим алгоритм управления (рисунок 13) согласно которому и будет происходить управление мощной нагрузкой.

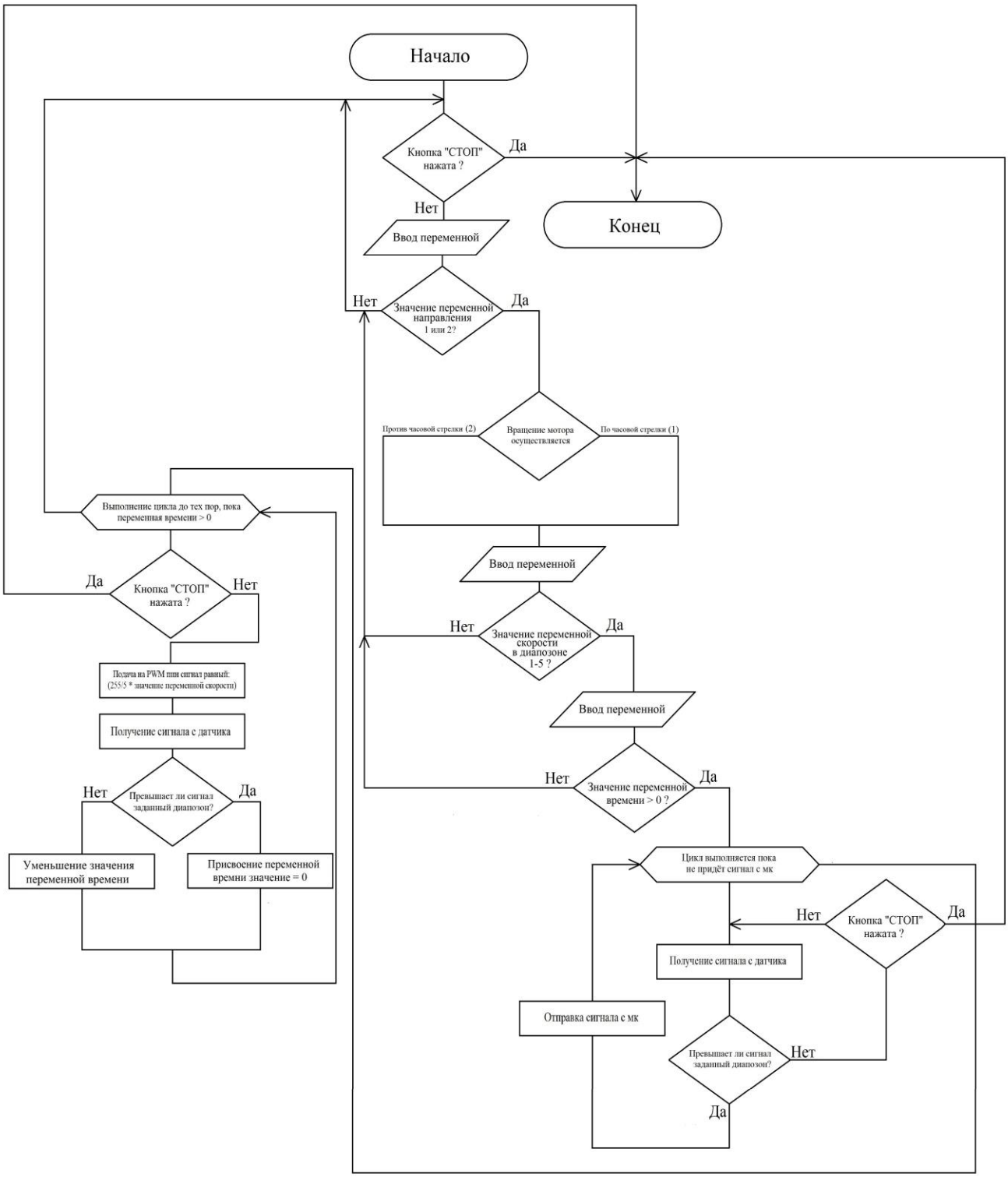

Рисунок 13 – Изображение алгоритма управления коллекторным электродвигателем

Организация связи LabVIEW & Arduino

Для того что бы организовать взаимоработу между нашим микроконтроллером и виртуальный прибор, роль которой выполняет среда разработки LabVIEW, необходимо прошить микроконтроллер с помощью скетча представленного на рисунке 14.

Скетч - это программа, написанная для платформы Arduino и имеющая определенную структуру.

```
// Включаем стандартные библиотеки в наш скетч
#include <Wire.h>
#include <SPI.h>
#include <Servo.h>
#include "LabVIEWInterface.h"
// Инициализируем Arduino и настраиваем последовательную связь
void setup()
{
  // Инициализация последовательного порта со скоростью передачи по умолчанию(9600)
   syncLV ();
}
// Постоянный цикл работающий в Arduino
// Цикл посылает и обрабатывает команды полученые от LabVIEW.
void loop()
{
  // Организуем условие отправки сигнала в LabVIEW
if (analogRead(A5) > 106) //Если полученный с датчика сигнал превышает заданный диапозон(106)..
{
 Serial.println(107);// Посылаем в монитор порта сигнал (107)
 }
 //Получение и обработка полученных команд полученных от LabVIEW
  checkForCommand();

  if(acqMode==1)
  {
    sampleContinously();
  }

}
```

Рисунок 14 – Скриншот скетча прошивки Arduino.

Приём и обработка данных полученных с микроконтроллера

После того как мы прошили наш микроконтроллер с помощью скетча представленного на рисунке 14, организуем приём сигнала с монитора порта Arduino (Рисунок 16) с помощью компонентов представленных на рисунке 15.

Рисунок 16 – Изображение используемых компонентов [3]

где:

а - VISA Configure Serial Port: устанавливает серийный порт к которому подключён микроконтроллер;

б - VISA Read: считывает данные с серийного порта подключенного устройства;

в - VISA Close: закрывает подключения;

г - Bytes at Serial Port: проверяет, доступны ли какие-то данные;

д - Fract/Exp String To Number Function: Преобразовывает полученную стринговую величину в числа.

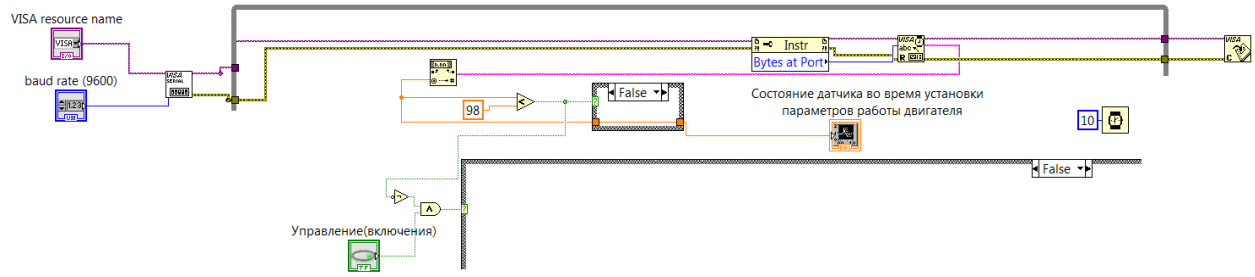

Рисунок 16 – Скриншот структуры программы реализующий приём сигналов

На рисунке 16 изображена структура приёмки и обработки сигналов полученных с микроконтроллера.

Устанавливаем связь порта используя компонент VISA Configure Serial Port, при поступлении сигнала с последовательного порта компонент Bytes at Serial Port посылает сигнал на VISA Read который преобразует полученный сигнал в величину типа String, для отображения данного сигнала ,и использования его для управления работы мотором, преобразовываем его в числовую величину с помощью компонента Fract/Exp String To Number Function. Если полученный сигнал не превышает заданную величину, булевая операция сравнения посылает сигнал на управляющий элемент цикла Case. На рисунке 17 показаны состояния при которых сигнал превышает и не превышает заданную величину. “Истинное” состояние управляющего элемента выполняет запуск цикла при котором графический индикатор отображает число 98, при “Ложном” состоянии на

графическом индикаторе будет отображается полученная с микроконтроллера величина (Рисунок 18). Так же если сигнал в “Ложном” состоянии, то после прохождения через инвертор примет “Истинное” состояние , и если была зажата кнопка ”Управление (включения)” на управляющий элемент цикла Case (Рисунок 19) будет подан соответствующий сигнал.

Стоит отметить что задержка ставиться лишь для того, что бы можно было более чётко наблюдать за изменением нашего сигнала.

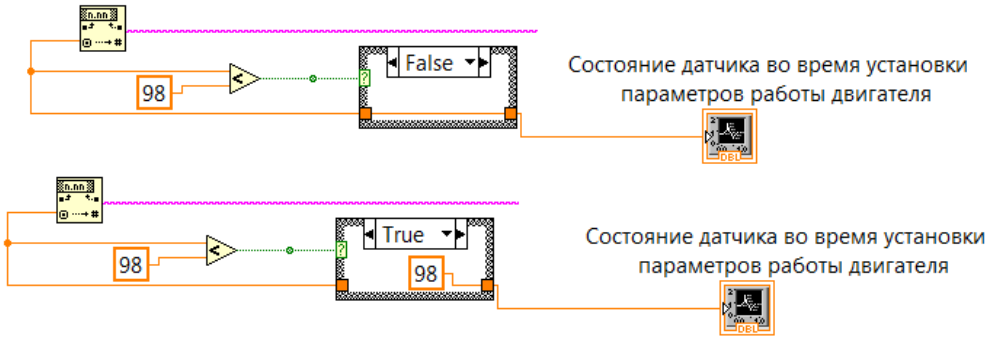

Рисунок 17 – Скриншот структуры отображения полученного сигнала

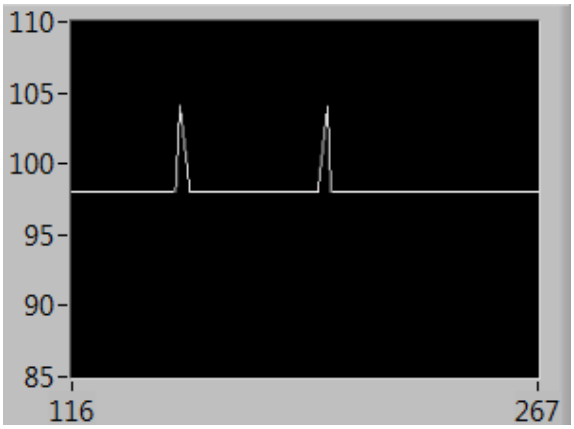

Рисунок 18 – Скриншот графического индикатора во время приёма сигнала

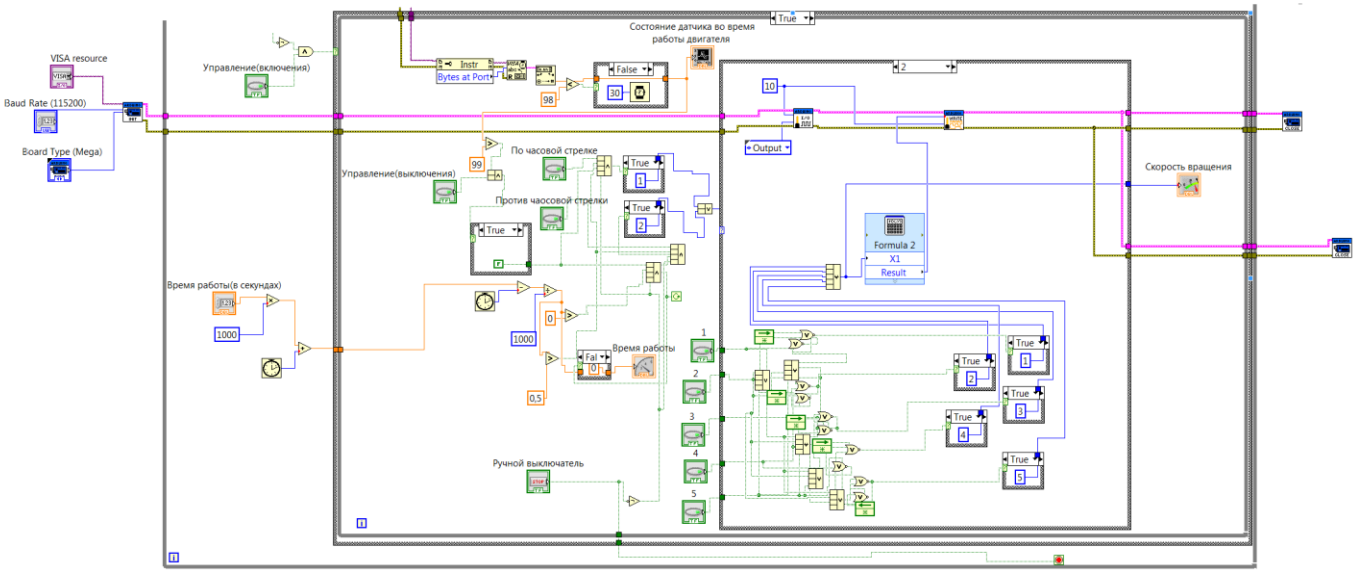

Рисунок 19 – Скриншот цикла Case организующего управление работой двигателя

Установка параметров работы коллекторного электродвигателя

Но прежде чем запускать работу нашего электродвигателя мы должны задать ему соответствующие параметры работы.

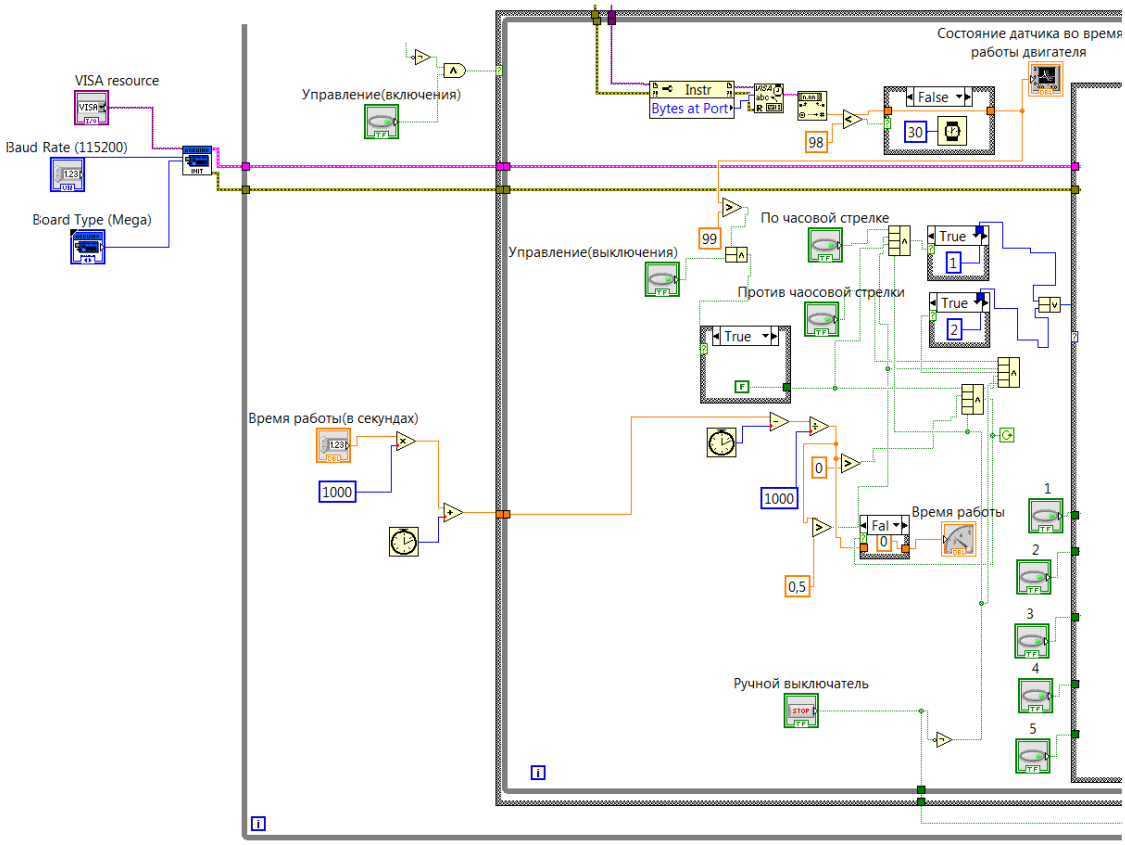

Рисунок 20 – Скриншот структуры организации управления параметрами работы электродвигателя

На рисунке 20 изображена структура организации управления работы электродвигателя, которая позволяет нам:

- задавать время работы нашего цикла и отображать его (при этом, когда значение таймера времени будет меньше 0,5 останавливаете работа мотора) (рисунок 21);

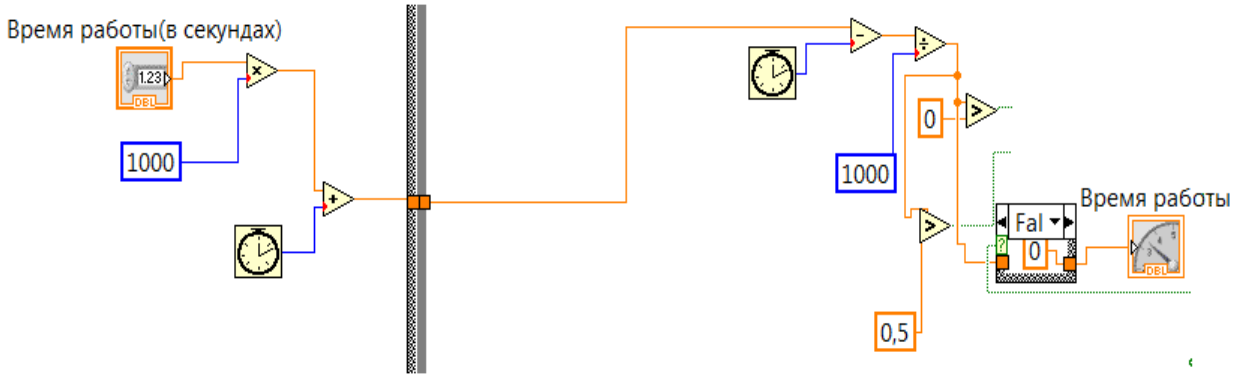

Рисунок 21 – Скриншот структуры организации управления временем работы электродвигателя

- выбирать направление вращения электродвигателя (рисунок 22).

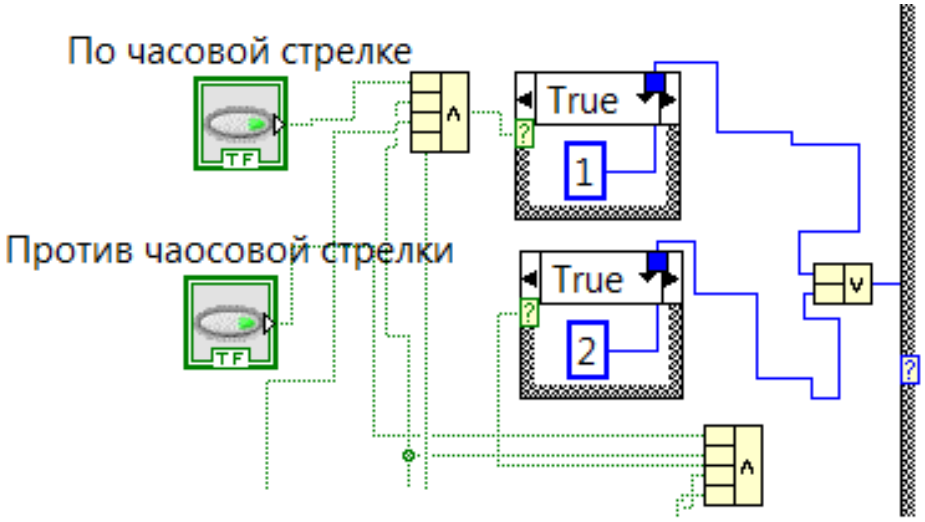

Рисунок 22 – Скриншот структуры организации управления направления вращения электродвигателя

- устанавливать скорость вращения электродвигателя (использования схему RS – триггера на элементах “И-НЕ” (рисунок 23) устанавливаем защиту от случайных нажатий (когда нажимаются 2 и более кнопки одновременно), тем

самым обеспечиваю свободную регулировку скорости с помощью одного нажатия на необходимую нам кнопку) (рисунок 24).

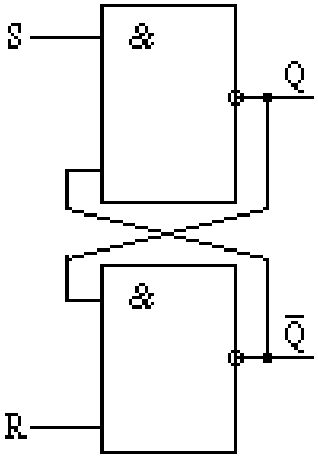

Рисунок 23 – Изображение RS- триггера на элементах “И-НЕ”

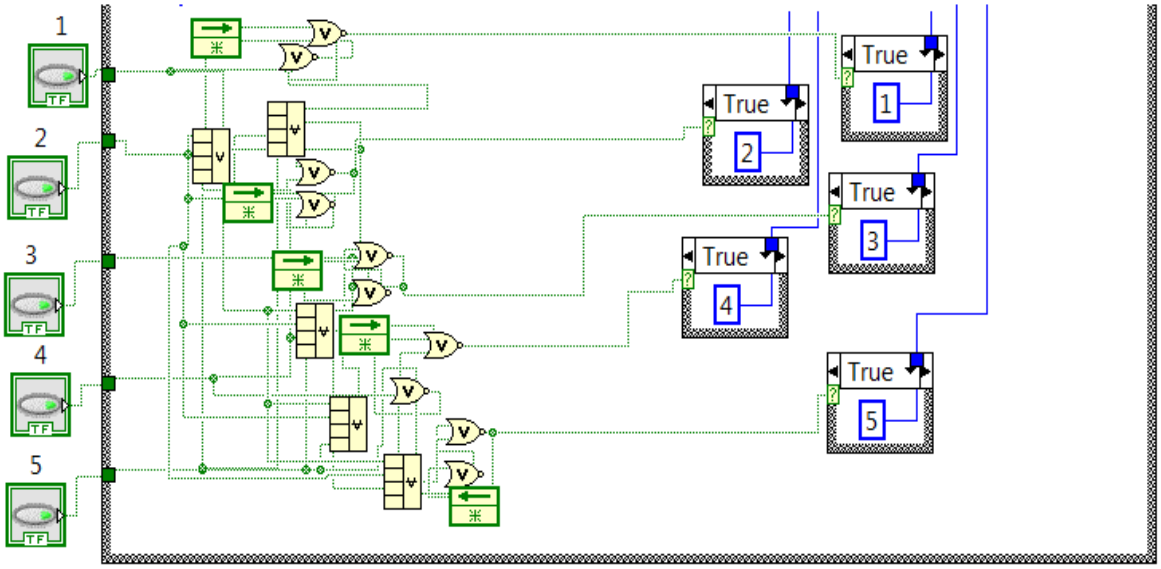

Рисунок 24 – Скриншот структуры организации управления скоростью вращения электродвигателя

- так же присутствует уже знакомая нам структура получения сигнала с мк, только в данном случае данная структура останавливает работу нашего цикла, тем самым останавливая работу двигателя. В случае если сигнал превысит заданное значение и будет зажата кнопка ”Управление (выключения)” цикл Case останавливает работу электродвигателя (рисунок 25).

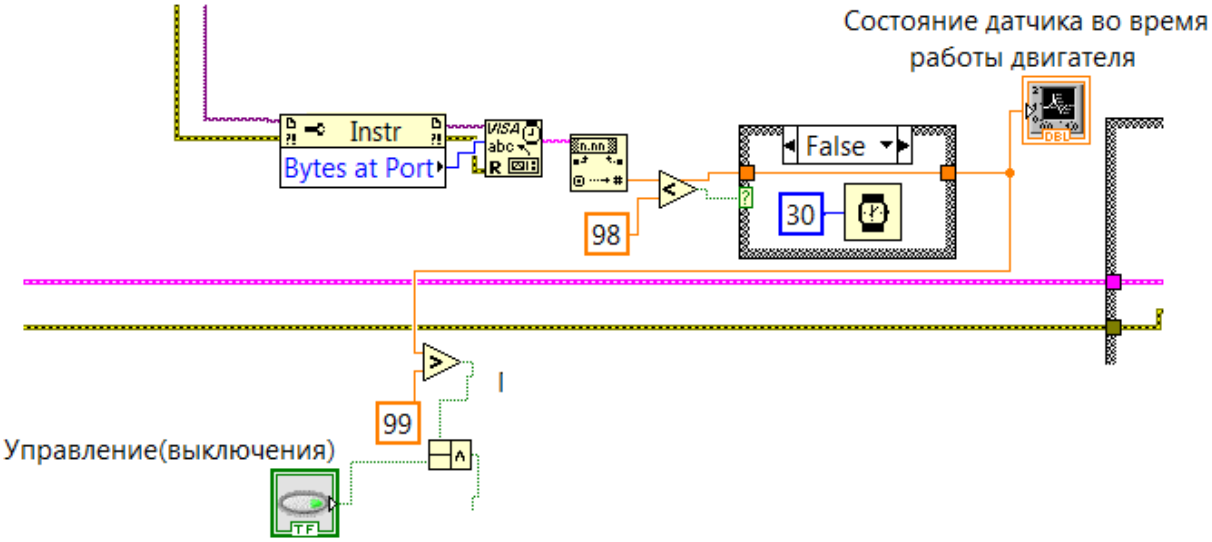

Рисунок 25 – Скриншот структуры программы реализующий остановку работы цикла

На протяжении работы всего цикла прослеживается состояние кнопки “Ручной выключатель” тем самым, давая возможность в любой момент остановить работу цикла сбрасывая все параметры работы коллекторного электродвигателя.

Организация подачи сигналов на микроконтроллер

Задав параметры работы электродвигателя и запустив цикл его работы мы посылаем сигналы на PWM пины нашей отладочной платы (рисунок 27) используя компоненты представленные на рисунке 26.

Рисунок 26 – Изображение используемых компонентов

где:

- а - Arduino Unit: Инициализирует соединение Arduino и LabVIEW;

- б - Set Digital Pin Mode :Устанавливает пин как выход или вход;

- в - PWM Write Pin:Передаёт на PWM пин сигнал (от 0 до 255 (0 - 5Вольт));

Широтно-импульсная модуляция или ШИМ (PWM) - это метод получения аналоговых значений с помощью цифровых средств. Цифровое управление используется для создания прямоугольной волны, сигнал имеет состояние 0 и 1.

- г - Arduino Close: закрывает подключение.

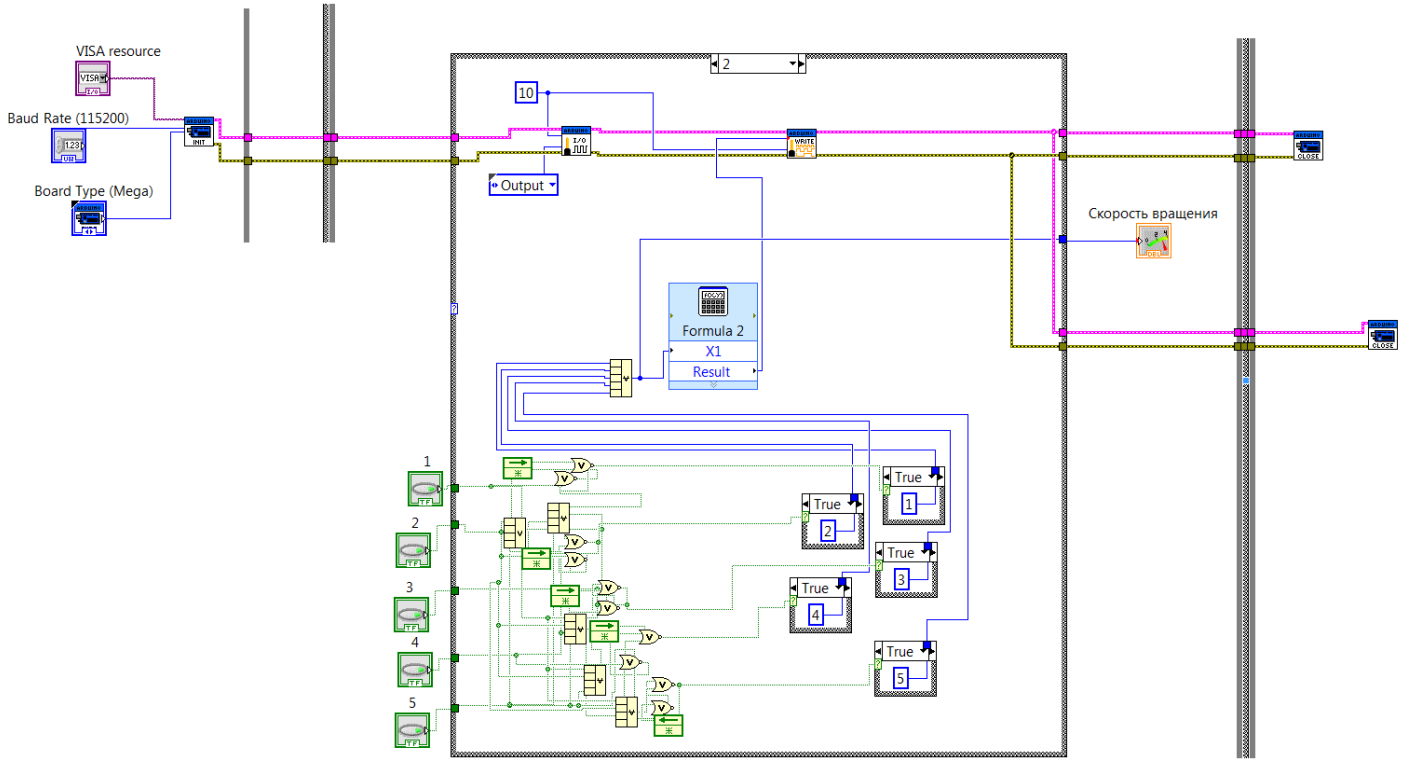

Рисунок 27 – Скриншот структуры посылания сигнала на PWM пины отладочной платы Arduino

Нажимая на кнопку от 1 до 5 мы посылаем на управляющий компонент цикла логическую единицу, тем самым приводя цикл в “Истинностное” состояние, который посылает соответствующее число на элемент PWM Write pin. Для того что бы подавать на PWM пин количество вольт равное отправляемому числу, сигнал из цикла Case проходит через компонент “Formula” и переводит число в число соответствующему числу вольт (рисунок 28).

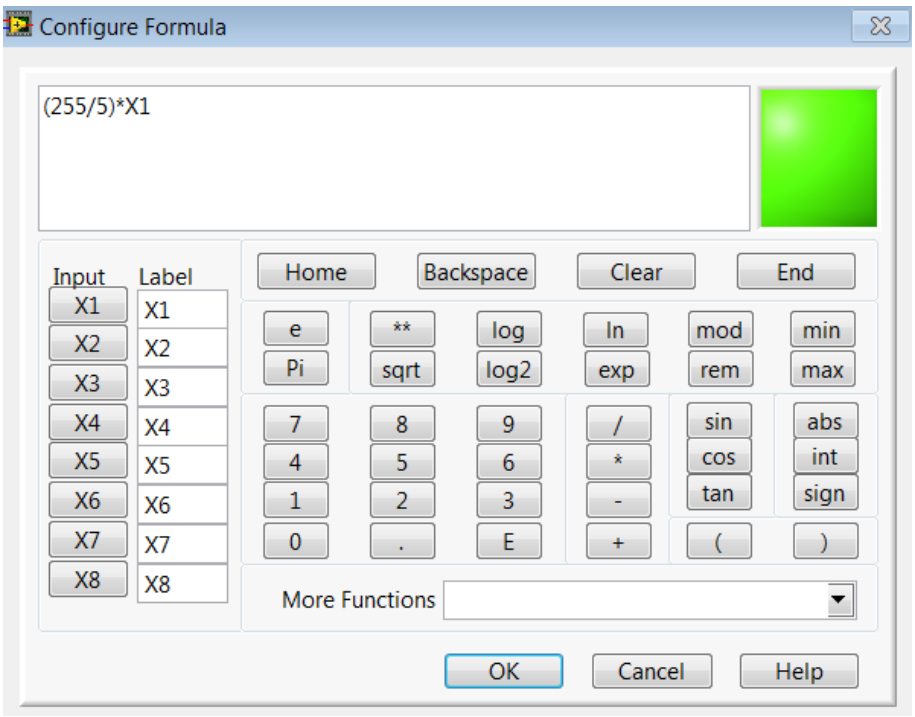

Рисунок 28 – Скриншот компонента “Formula”

Так же в зависимости от выбранного направления вращения мотора зависит пин на который подаётся напряжение (Рисунки 29, 30).

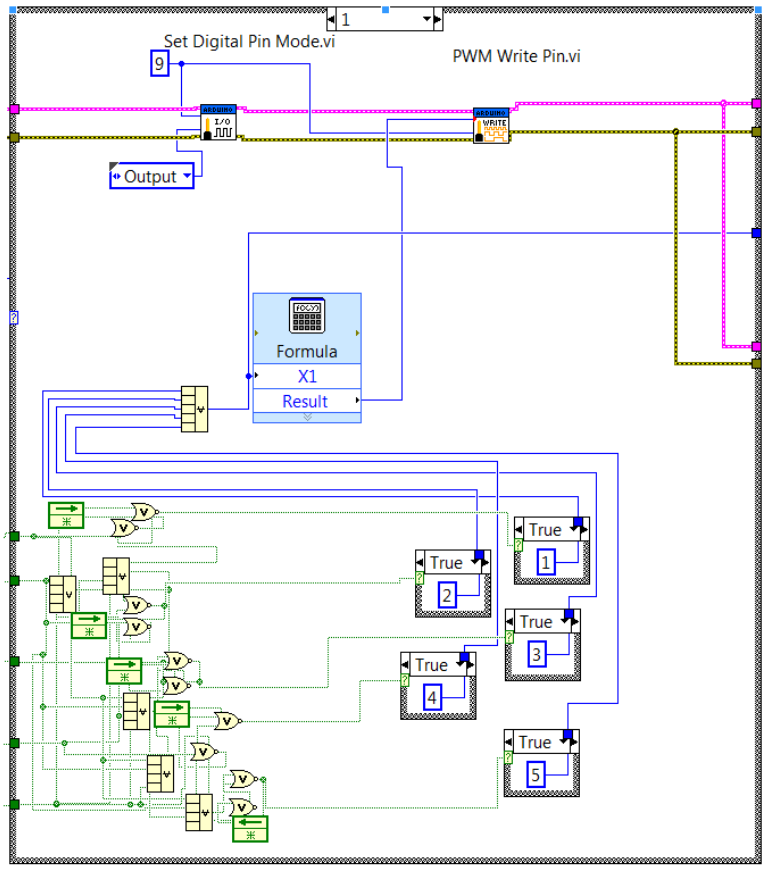

Рисунок 29 – Скриншот структуры отправления сигнала на PWM пины при вращении двигателя по часовой стрелки

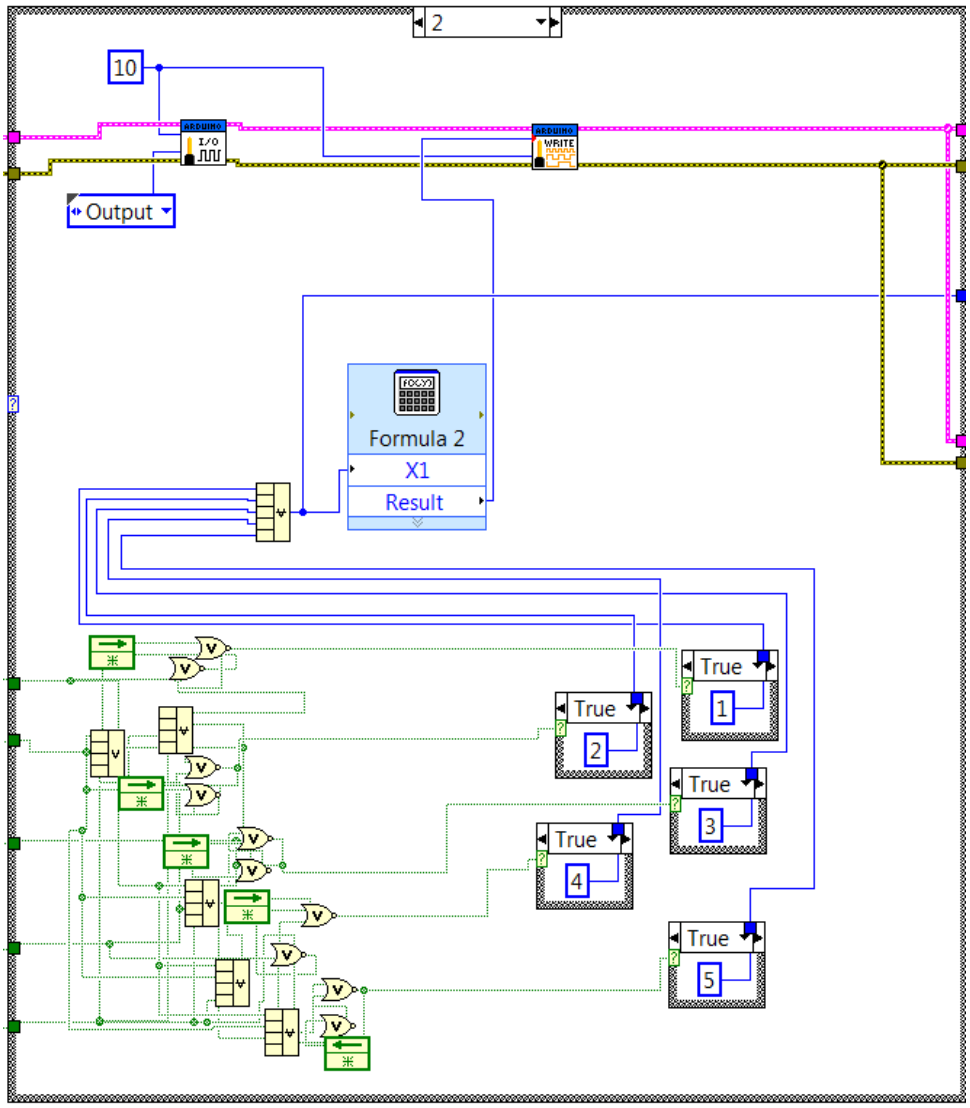

Рисунок 30 – Скриншот структуры отправления сигнала на PWM пины при вращении двигателя против часовой стрелки

Стоит отметить, что если в случае не выбранного направления, случайного нажатия двух кнопок одновременно, нажатии кнопки “Ручной выключатель” или завершении времени работы электродвигателя, цикл переходит в режим “Бездействия”, при котором на оба пина посылается число 0, который приводит систему в режим “Ожидание задания параметров работы” (Рисунок 31).

Подаваемое количество Вольт на пин отображается на индикаторе представленного на рисунке 27.

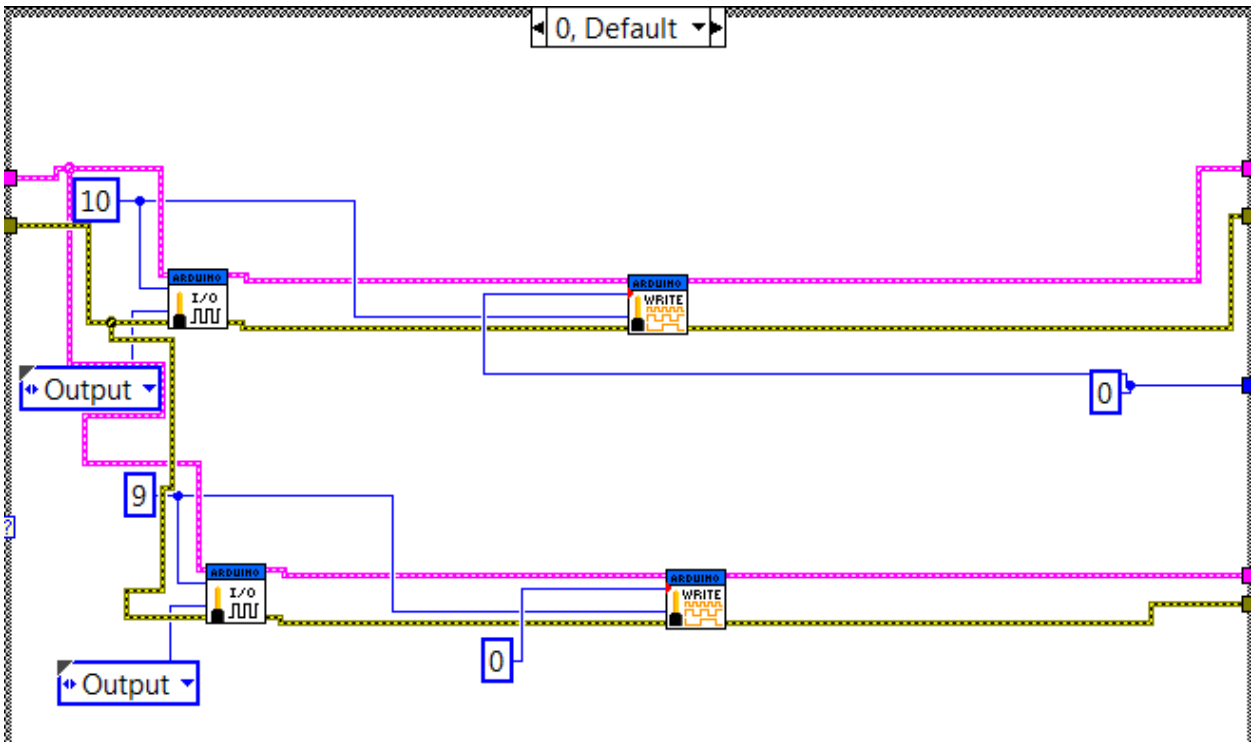

Рисунок 31 – Скриншот структуры отправления сигнала на PWM пины во время режима “Бездействие”

Front Panel и Block Diagrama

Для такого что бы мы могли устанавливать параметры работы электродвигателя необходимо открыть окно Front Panel (Рисунок 32).

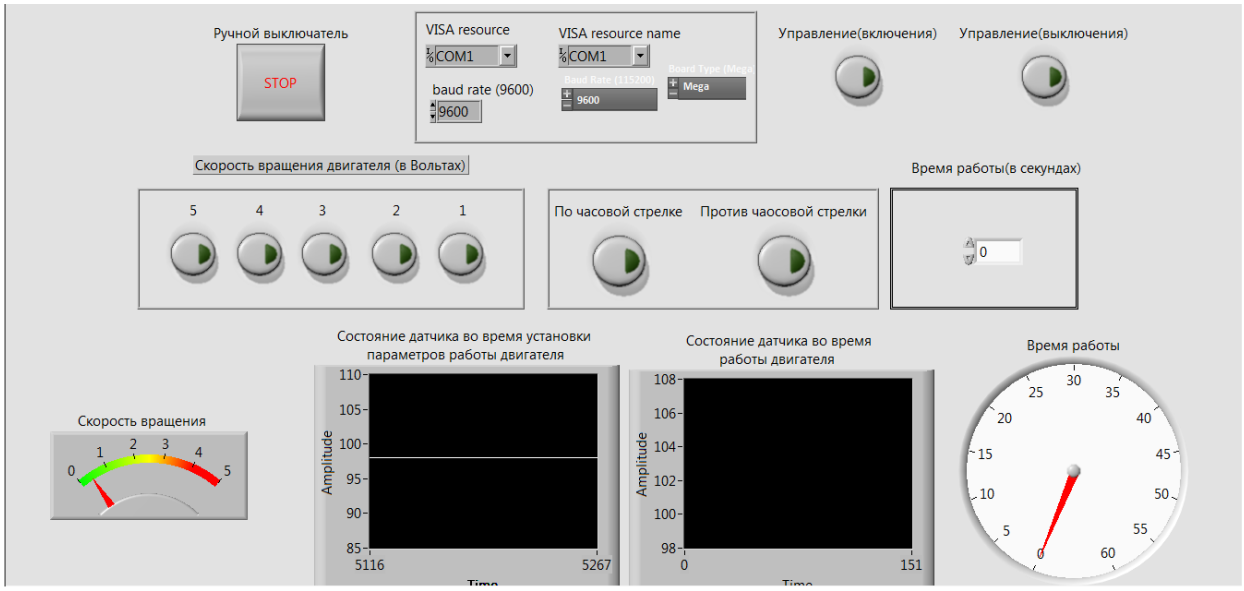

Рисунок 32 – Скриншот окна Front Panel.

На рисунке 32 , мы видим всю представленную нам работу системы и управляющие элементы:

- Окно настройки инициализации Arduino для задания типа платы, номера последовательного порта, частота работы порта (Рисунок 33) ;

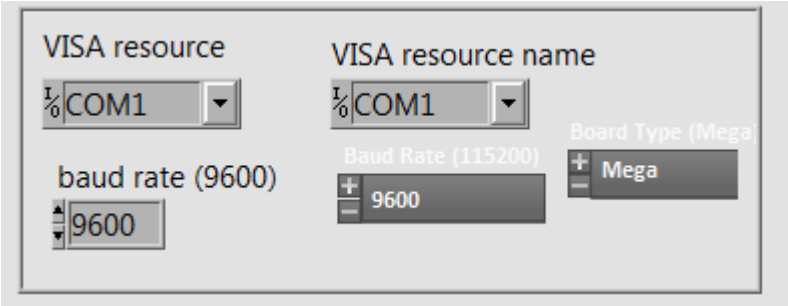

Рисунок 33 – Скриншот окна настройки инициализации Arduino.

- Окно настройки параметров работы электродвигателя (Направление вращения двигателя, скорость вращения, время вращения) (рисунок 34);

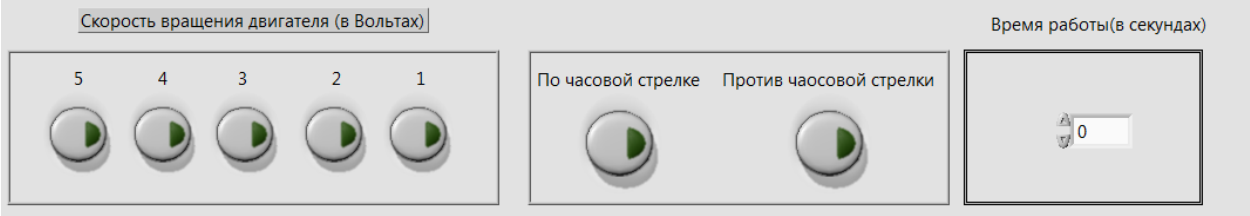

Рисунок 34 – Скриншот окна настройки параметров работы электродвигателя.

- Окно управления включения и выключения цикла работы двигателя (Рисунок 35);

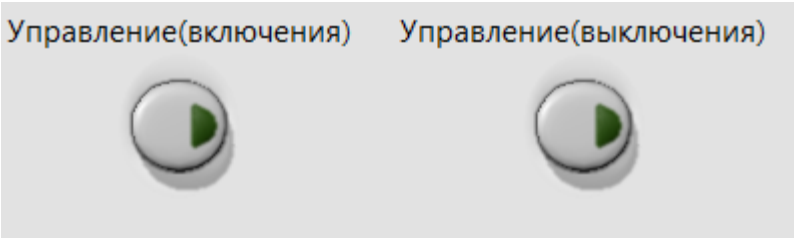

Рисунок 35 – Скриншот окна управления включения/выключения работы электродвигателя.

- Индикаторы, показывающие состояние работы электродвигателя и полученные с датчика сигналы (Рисунок 36).

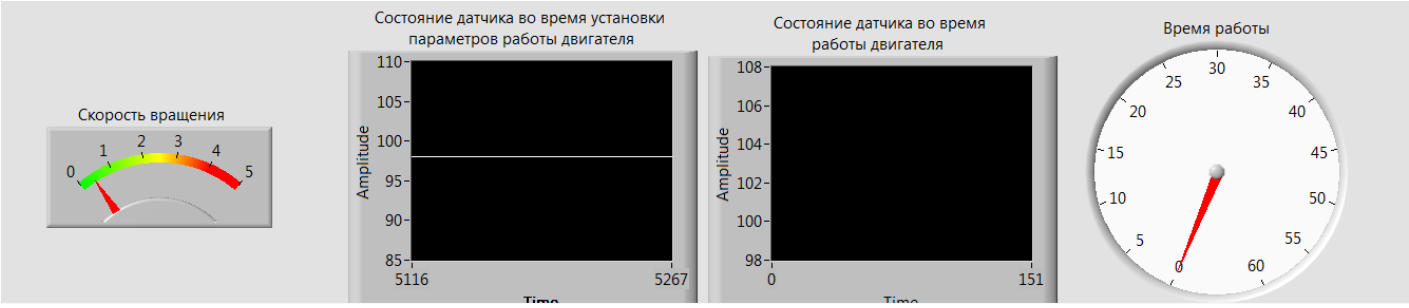

Рисунок 36– Скриншот окна индикаторов.

Изображение программы выполненной в окне Block Diagrama представлено на рисунке 37.

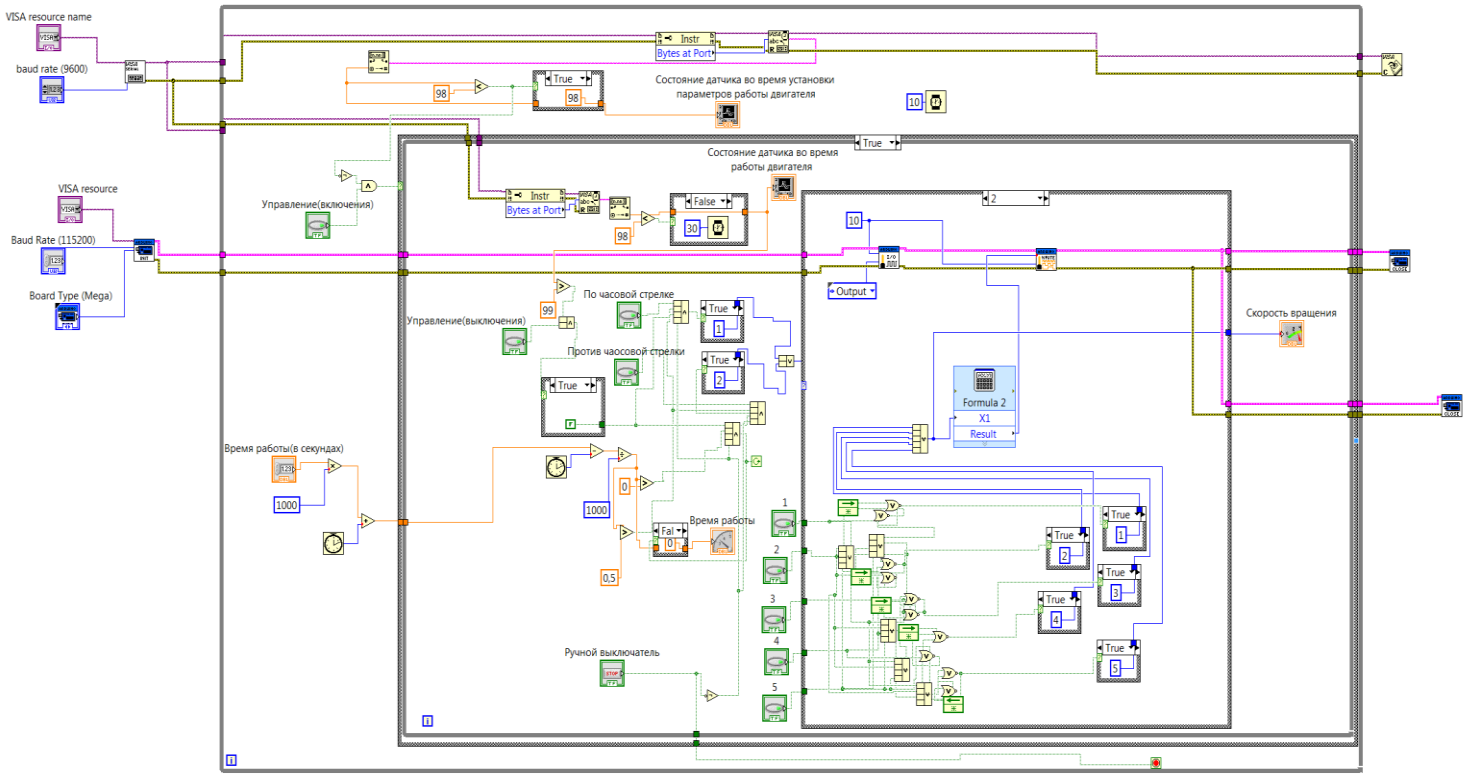

Рисунок 37 – Скриншот программы выполненной в окне Block Diagrama.

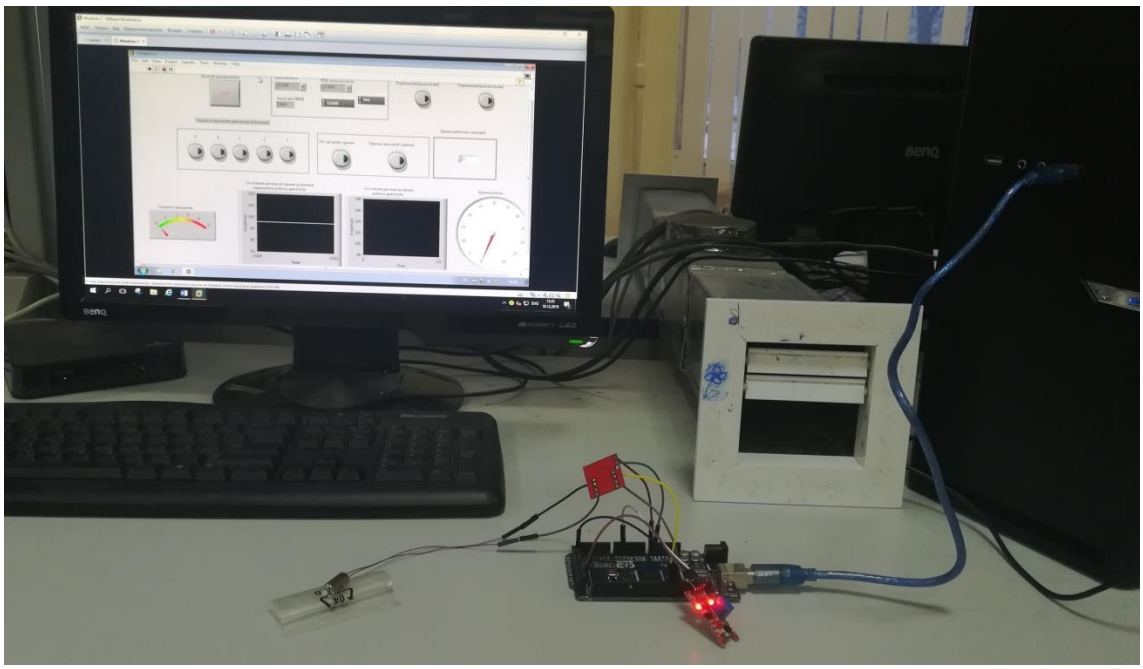

Рисунок 38 – Фотография установки перед запуском электродвигателя.

Выводы по программе

К **плюсам** данной программы можно отнести:

- Простое и понятное управление с помощью окна Front Panel;

- Логичность программы;

- Возможность изменения параметров работы электродвигателя в режиме реального времени;

- Включение и остановка работы мотора требует зажатой кнопки “Управления” (Рисунок 192), для предотвращения получения случайного сигнала с датчика.

Однако присутствуют и **минусы**:

- Корректная работа программы зависит от вычислительной мощи ПК;

- Индикатор времени работает в пределе до 60 секунд;

- Необходимо чётко настроить датчик шума, для корректного управления системой;

Альтернативный вариант реализации системы

Альтернативным вариантом реализации управления коллекторным электродвигателем является использование только программного обеспечения Arduino IDE.

Структурная схема подключения электродвигателя

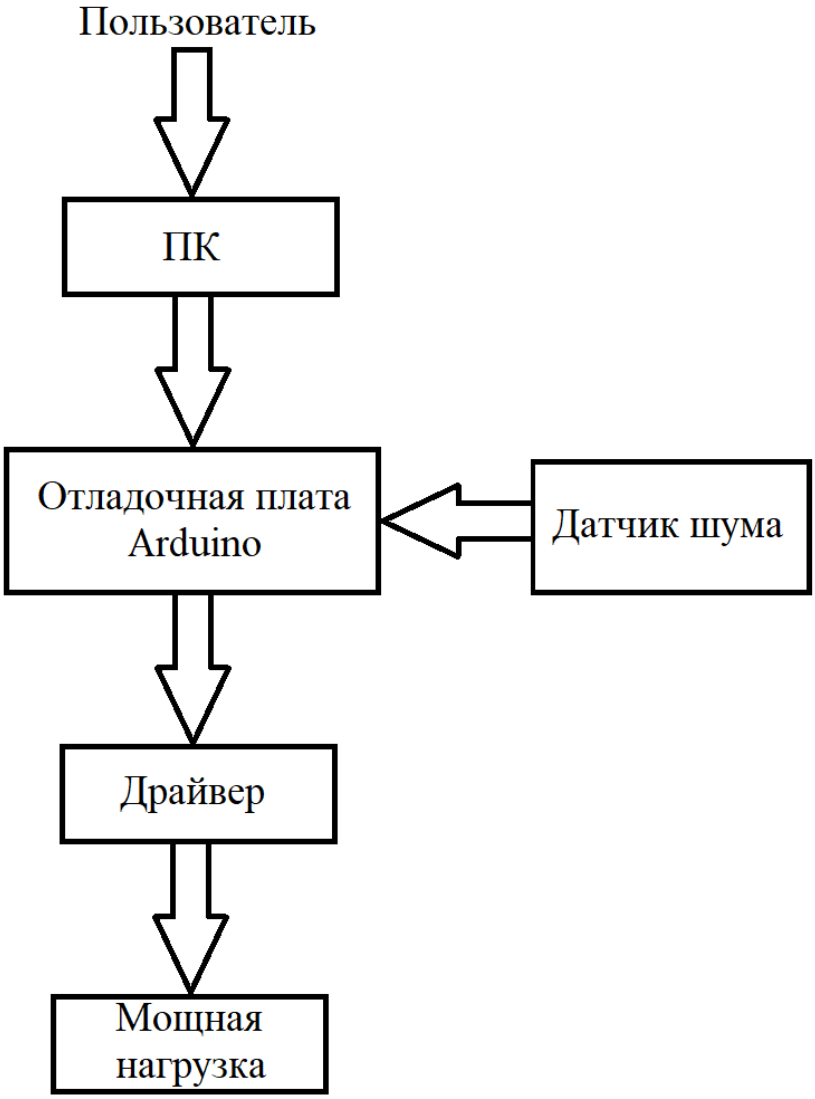

Рисунок 39 – Изображение структурной схемы подключения коллекторного электродвигателя

На основе составленной структурной схемы подключения коллекторного электродвигателя (Рисунок 39), составим программу, используя программное обеспечение Arduino IDE, реализации управления коллекторным электродвигателем .

```
#define a 9
#define b 10
byte incomingByte;
int x,y,r,s,t,q,w;
int j=1000;
float l;

void setup()
{
Serial.begin(19200);
pinMode(2,INPUT);
pinMode(a,OUTPUT);
pinMode(b,OUTPUT);
// Вывод инструкции реализации управления нагрузкой
Serial.println();
Serial.println("Введите направление вращения мотора(1-по часовой; 2-против часовой)");
Serial.println("Введите направление скорость вращения мотора,в вольтах поданых на питание мотора(от 1 до 5)'
Serial.println("Введите время работы мотора(в секундах)");
Serial.println("_____________________________________");
Serial.println();

 }

void loop()
{
  // Установка цикла While для поочерёдного ввода параметров работы мотора
   while(x<3)
   {
     if ((Serial.available() > 0) && (x==0)) // Проверка состояния порта на наличие входных данных
   {
    incomingByte = Serial.read();// Чтение значения с монитора порта

    if(incomingByte == '1')
    {
    r=1; // Запись значения переменной для осуществления управления направление работы мотора
    Serial.println("Направление мотора осуществляется по часовой стрелке");
    }
    else if (incomingByte == '2')
      {
          Serial.print("Направление мотора осуществляется против часовой стрелке");
     r=2;
      }
      // В случае некорректного задания параметра выдать отчёт по ошибке
     else
      {
    Serial.println("Мне что вверх лететь?");
      }
    if (r>0)
      {
    Serial.flush();
    x=1;
      }
   }
```

Рисунок 40 – Скриншот начала программы

```
    if ((Serial.available() > 0) && (x==1)) //Повтор предыдущего цикла для задания параметра скорости вращения
    {
    incomingByte = Serial.read();
    if(incomingByte == '1')
    {
     s=1;
    }
    else if (incomingByte == '2')
    {
     s=2;
    }
    else if (incomingByte == '3')
    {
    s=3;
    }
    else if (incomingByte == '4')
    {
     s=4;
    }
    else if (incomingByte == '5')
    {
    s=5;
    }
    else
    {
    Serial.println(")");
      }
if (s>1)
{
      Serial.println();
        Serial.print("На мотор будет подано следущее кол-во Вольт: ");
        Serial.print(s);
  x=2;
Serial.flush();
}
    }
 if ((Serial.available() > 0) && (x==2)) //Повтор предыдущего цикла для задания параметра времени работы вращения мотора
 {
t = Serial.parseInt();
 x=3;
            Serial.println();
      Serial.print("Мотор будет работать в течении " );
      Serial.print(t);
Serial.print(" секунд");
Serial.println();
      l=t;
  }
    }
  if((analogRead(A5)>106) && (x==3))//Опрос состояния датчика шума (если "Шум" превышает значение 106) и заданны параметры работы ...
  {
    Serial.println("WORKING...");// Вывод строки о подаче на мотор заданое количесвто Вольт.
  delay(300);
  while( ((float(r))==1) &&(l>=0.01)) // Пока переменная времени не станет ровнять 0.01 и был задан параметр вращения мотора по часовой стрелке...
  {
  digitalWrite(a,LOW); // Подача на один из входов мотора 0 вольт
  int k=((255/5)*s);
  analogWrite(b,k); // подача на другой вход мотора заданное количество вольт
  l=l-0.005;
  q=analogRead(A5);
  delay(3);
  w=analogRead(A5);
  if ((q>106) && (w>106)) // Если длительность превышения уровня шума больше 3мс
  {
  l=0;// время работы мотора становится равноё 0
  }
  delay(2);
  }
  while( ((float(r))==2) &&(l>=0.01)) // Аналогичный цикл,при заднии параметра вращения мотора против часовой стрелки
  {
  Serial.println("WORKING...");
  int k=((255/5)*s);
  analogWrite(a,k);
  q=analogRead(A5);
  delay(3);
  w=analogRead(A5);
  if ((q>106) && (w>106))
  {

  l=0;
  }
  l=l-0.005;
  delay(2);
  }
  Serial.println("Stop");// Сообщение об остановке работы
  //подача на мотор 0В для остановки его работы
  digitalWrite(b,LOW);
  digitalWrite(a,LOW);
  x = 0; // Сброс переменной для организации повторного ввода данных
  Serial.println("Повторите процедуру ввода данных");
  }
    }
```

Рисунок 41 – Скриншот конца программы

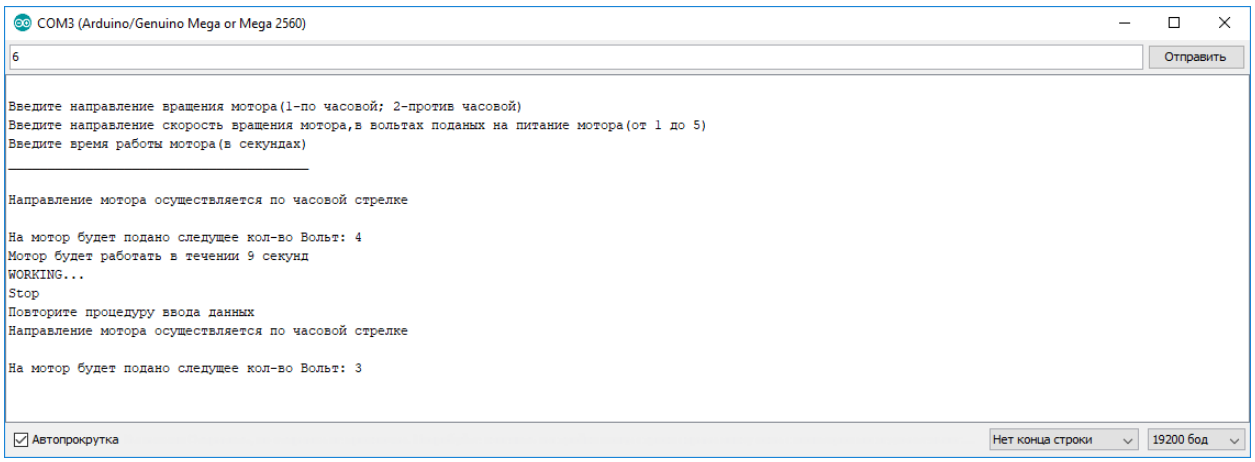

Рисунок 42 – Скриншот Монитора последовательного порта Arduino во время работы.

Благодаря использованию только ПО Arduino IDE система получает возможность работать автономно (без использования ПК), однако в данном случае мы используем ПК для ввода параметров работы электродвигателя через последовательный монитор порта